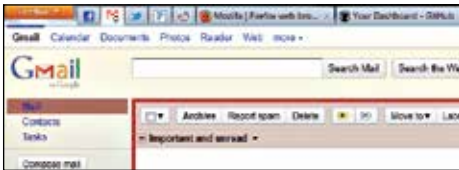


```

• <code>
•
• function triggerOnLocationChange(event) {
•   let win = event.target.ownerDocument.defaultView;
•   win.XULBrowserWindow.onLocationChange(
•     {DOMWindow: win.content}, /* stub aWebProgress obj */
•     null,
•     win.gBrowser.selectedTab.location
•   );
• }
•
• var delegate = {
•   onTrack: function (window) start(window),
•   onUntrack: function (window) stop(window)
• }
•
• new wu.WindowTracker(delegate);
•
• </code>

```

The code is pretty and functional as well. We see that WindowTracker is used from the window-utils API, and that we use the 'start' function defined earlier for tracked entities and 'stop' for untracked, and an event



Aaapptabs in action. Neat!

listener is applied to track any changes in state. The event listener would report any changes that need to be made (whether or not the chrome has to be altered) when the selected tab is changed.

The purpose of this add-on is removing the names of tabs, which hold apps, thus giving you more screen real estate with your currently open tab. Of course, you'll have to use this feature to appreciate it.

Now, your code should ideally be different, one that implements your own idea. So have fun creating your own! 